AMERICAN UNIVERSITY OF BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

[CSC 2210]: OBJECT ORIENTED PROGRAMMING 2

[SUMMER 25-26]
Section: [H]

Project Report On

Cafe Management System

Submitted By:

Name	ID
1. MIR MASNUN BIN ZAMAN	24-56830-1
2. SK SOAIB RISAN	24-56442-1
3. MD. MAHMUDUL HASAN MISSAL	24-56726-1
4. SAMIA ISLAM	23-53031-3

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				

Table of Contents

1. Introduction3
2. Feature List3
    2.1 Admin3
    2.2 Cashier3
    2.3 Customer3
3. ER Diagram4
    Normalization4
    Finalization5
4. UML Diagram6
    4.1 Use Case Diagram6
    4.2 Activity Diagram6
5. Conclusion7
6. UI Screenshots9

1. Introduction

Cafes and small food-service businesses handle a high volume of short, repetitive transactions — taking an order, calculating a total, collecting payment, and tracking stock — many times per hour. Managing these operations manually with paper tickets and a cash drawer makes it difficult to track daily sales accurately, monitor stock levels in real time, and hold individual staff accountable for the transactions they process.

BrewPoint is a desktop-based Cafe Management System developed in C# using the WinForms framework, backed by a SQL Server database accessed through ADO.NET. The system is designed around two authenticated staff roles — an Admin who manages the menu and staff accounts, and a Cashier who operates the point-of-sale (POS) screen — while the walk-in Customer remains a central actor in the real-world workflow even though they do not hold a system account.

The system automates the full sales cycle: browsing an up-to-date menu, building an order, calculating totals and change, committing the completed sale to the database as an atomic transaction, and automatically decrementing stock. It also gives each role a dashboard suited to their responsibilities — store-wide statistics for the Admin, and personal sales performance for each Cashier.

The goal of this project is to demonstrate the practical application of Object-Oriented Programming concepts — encapsulation, abstraction, polymorphism, and inheritance — within a real, functioning, database-integrated desktop application, while also applying sound relational database design principles including normalization and transactional data integrity.

2. Feature List

2.1 Admin

The Admin is the highest-level user of the system, responsible for the data that the rest of the system depends on. The Admin manages the full menu — adding, updating, and deleting products, along with their category, price, stock level, and availability — and manages staff accounts, including creating new Cashier logins, updating their details, and deactivating accounts that should no longer have access. The Admin also has access to a store-wide dashboard showing total orders, today's income, total income, and the number of active cashiers, giving an at-a-glance view of the business without needing to query the database directly.

2.2 Cashier

The Cashier operates the point-of-sale screen where the actual transaction with a customer takes place. This includes viewing the live, in-stock menu; selecting products and quantities into a running cart; calculating the order total and the change due from the amount tendered; and finalizing the sale, which commits the order and its line items to the database and decrements stock in a single all-or-nothing transaction. Each Cashier also has a personal dashboard showing their own sales for the day, progress toward a daily sales goal, and their all-time total sales — since the system supports multiple cashiers simultaneously, this data is scoped to whichever cashier is currently logged in, not the store as a whole.

2.3 Customer

The Customer is a walk-in actor who interacts with the system indirectly, through the Cashier, rather than holding a login account. A customer approaches the counter, places their order verbally, and pays the Cashier, who enters the order into the system on their behalf. This design choice was made deliberately: requiring a customer to create an account and log in just to purchase a coffee introduces unnecessary friction for what is typically a short, low-value, walk-in transaction. An earlier version of this project included a

self-registration screen for customers; it was intentionally removed from the final build in favor of this simpler, guest-style workflow that better reflects how real cafes operate at the counter.

3. ER Diagram

The system's data is organized around four entities: Users (staff accounts), Products (menu items), Orders (one row per completed transaction), and OrderItems (the individual line items within an order). Orders and Products have a many-to-many relationship — one order can contain many products, and one product can appear across many orders — which is resolved through the OrderItems junction table.

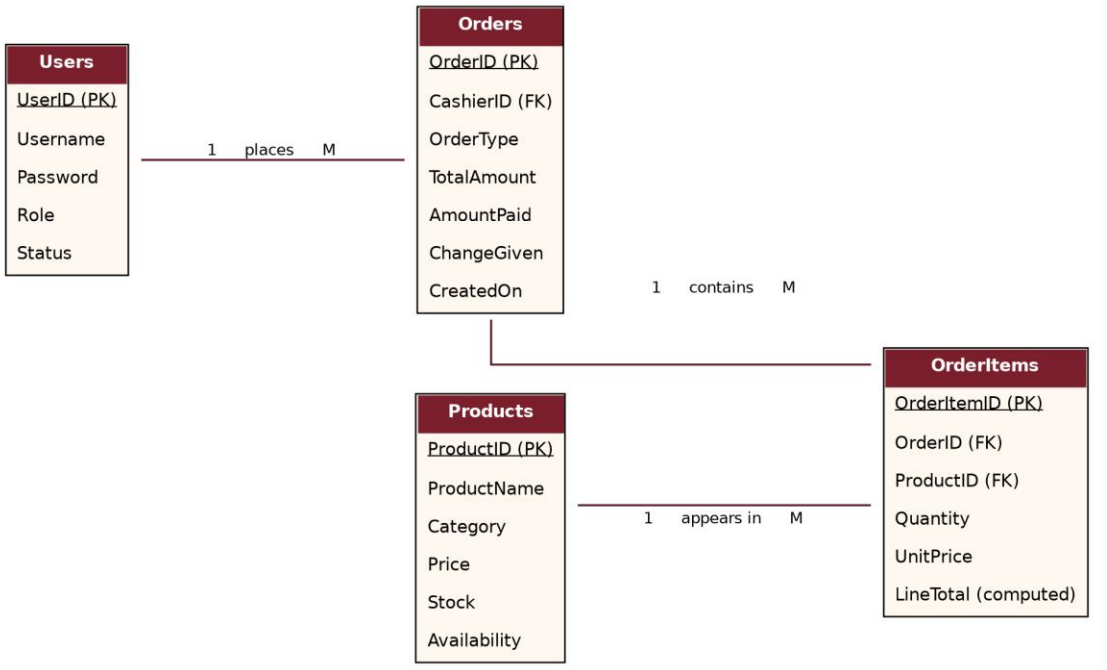


Figure 1: Entity-Relationship Diagram of the Cafe Management System

Normalization

The schema below is normalized in three stages for each relationship, following the same UNF → 1NF → 2NF process used to validate that every table's non-key attributes depend only on that table's own primary key.

1. Users & Orders (places)

UNF: UserID(PK), Username, Password, Role, Status, OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn

1NF: UserID(PK), Username, Password, Role, Status, OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn

2NF:

- 1. UserID(PK), Username, Password, Role, Status
- 2. OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn, UserID(FK)

2. Orders & OrderItems (contains)

UNF: OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn, OrderItemID(PK), Quantity, UnitPrice, LineTotal

1NF: OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn, OrderItemID(PK), Quantity, UnitPrice, LineTotal

2NF:

- 1. OrderID(PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn
- 2. OrderItemID(PK), Quantity, UnitPrice, LineTotal, OrderID(FK)

3. Products & OrderItems (appears in)

UNF: ProductID(PK), ProductName, Category, Price, Stock, Availability, OrderItemID(PK), Quantity, UnitPrice, LineTotal

1NF: ProductID(PK), ProductName, Category, Price, Stock, Availability, OrderItemID(PK), Quantity, UnitPrice, LineTotal

2NF:

- 1. ProductID(PK), ProductName, Category, Price, Stock, Availability
- 2. OrderItemID(PK), Quantity, UnitPrice, LineTotal, ProductID(FK)

Since no non-key attribute in any of the tables above depends on another non-key attribute (no transitive dependency exists — for example, Price depends only on ProductID, not on any other column in Products), each table also satisfies Third Normal Form (3NF) at this stage; no further decomposition is required.

Finalization

- 1. Users (UserID (PK), Username, Password, Role, Status)
- 2. Products (ProductID (PK), ProductName, Category, Price, Stock, Availability)
- 3. Orders (OrderID (PK), OrderType, TotalAmount, AmountPaid, ChangeGiven, CreatedOn, UserID (FK))
- 4. OrderItems (OrderItemID (PK), Quantity, UnitPrice, LineTotal, OrderID (FK), ProductID (FK))

4. UML Diagram

4.1 Use Case Diagram

The Use Case Diagram shows how the Admin, Cashier, and Customer interact with the system. The Admin logs in to manage products and staff accounts and to view store-wide statistics. The Cashier logs in to take orders, manage the cart, process payments, and view their own sales dashboard. The Customer does not hold a system account, but is shown interacting with the system indirectly through the Cashier — placing an order and receiving it once payment is processed — which reflects the guest-checkout design used throughout this project.

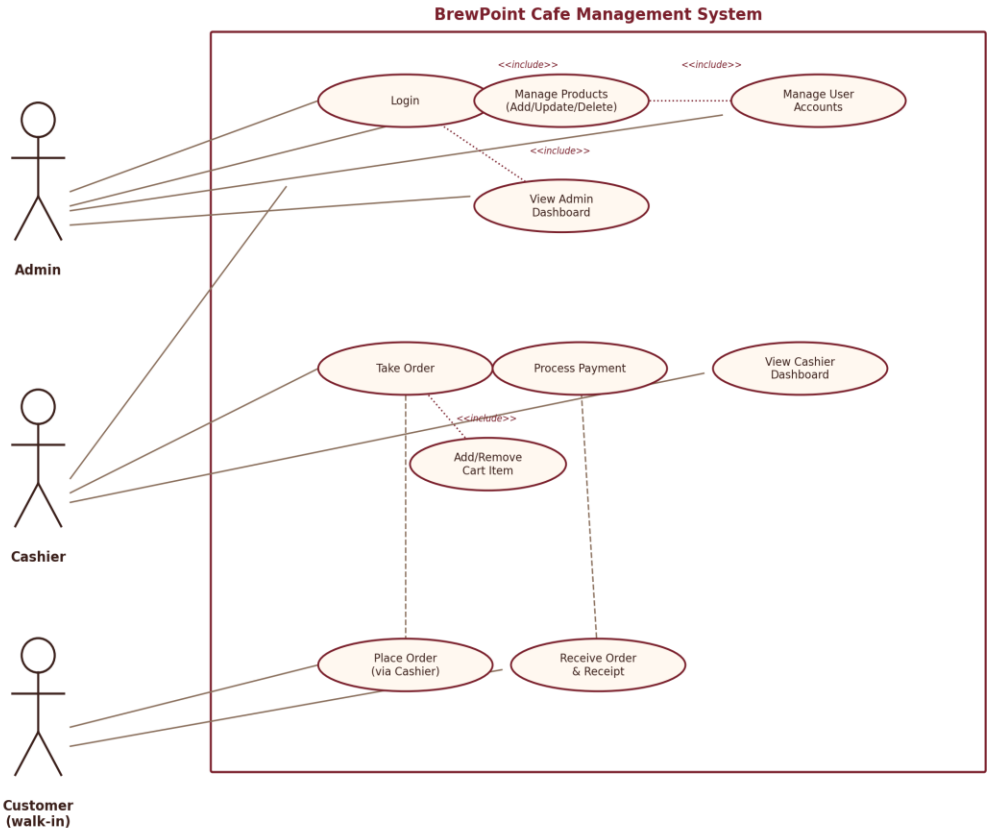


Figure 2: Use Case Diagram of the Cafe Management System

4.2 Activity Diagram

The Activity Diagram represents the full workflow of a single transaction from start to finish, across all three actors. It begins when the Customer approaches the counter and places an order; the Cashier logs in, selects the requested products into the cart, calculates the total, and confirms the amount tendered is sufficient before finalizing the sale. Finalizing the sale commits the order to the database as a single transaction and updates stock. In parallel, the Admin's workflow of managing products, managing user accounts, and reviewing the dashboard draws on the same order data that the Cashier's transactions produce.

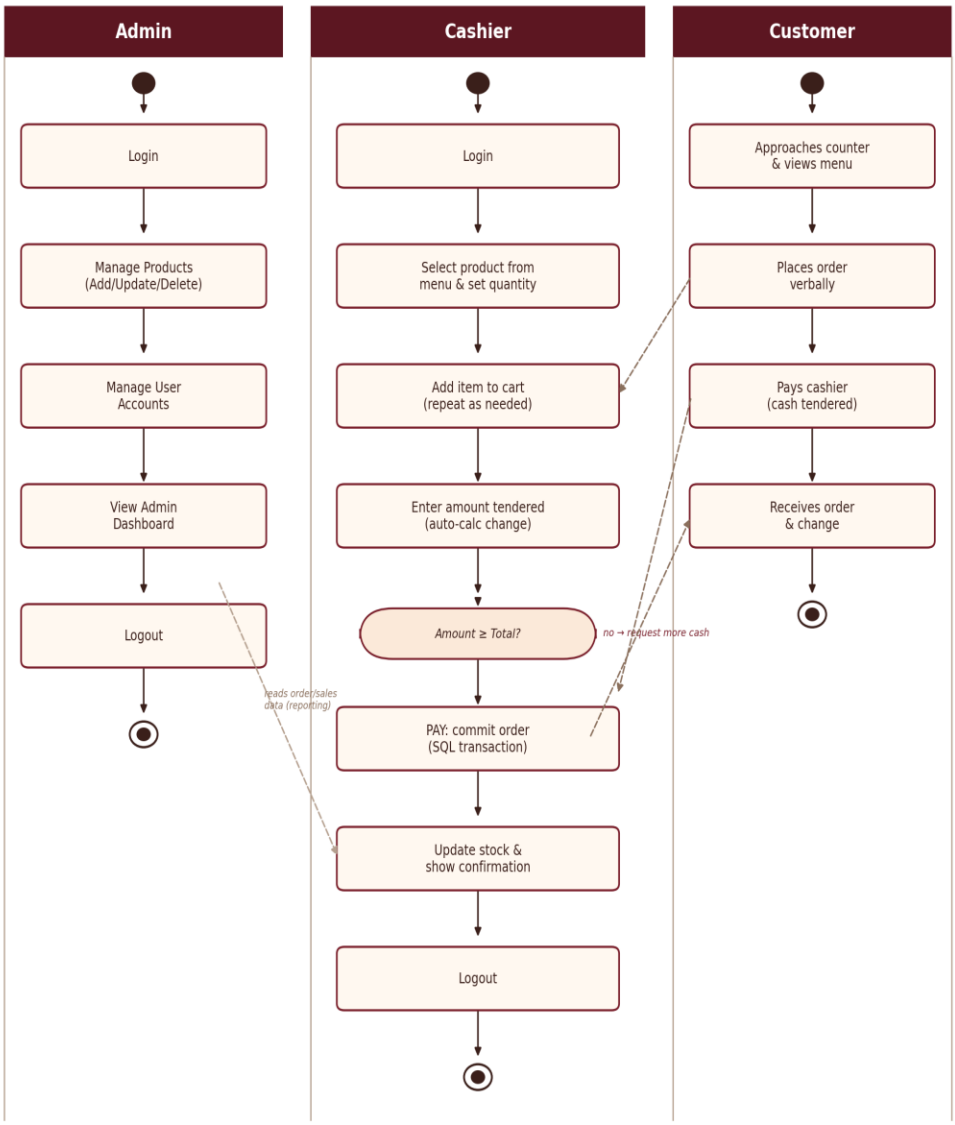


Figure 3: Activity Diagram of the Cafe Management System

5. Conclusion

The BrewPoint Cafe Management System provides a focused, working solution for the core daily operations of a small cafe: managing the menu, taking orders, processing payments, and tracking each cashier's performance. By separating the Admin and Cashier roles and scoping each dashboard to what that role actually needs to see, the system avoids overwhelming any one user with irrelevant information while still giving the Admin full oversight of the business.

Deliberately removing customer self-registration in favor of a guest-checkout workflow reflects a realistic understanding of how counter-service cafes actually operate, where speed and low friction matter more than account-based personalization for a typical walk-in purchase. The use of a database transaction when finalizing a sale ensures that an order, its line items, and the resulting stock deduction are always kept consistent, even if an error occurs partway through the process.

Overall, the project demonstrates the practical integration of an object-oriented desktop application with a normalized relational database, while highlighting realistic, documented trade-offs — such as plain-text password storage and a hardcoded daily sales goal — that were made consciously to keep the project's scope achievable within the available time, with clear paths identified for how each could be improved in a production system.

6. UI Screenshots

The following screenshots show the working application, captured directly from Visual Studio during development and testing.

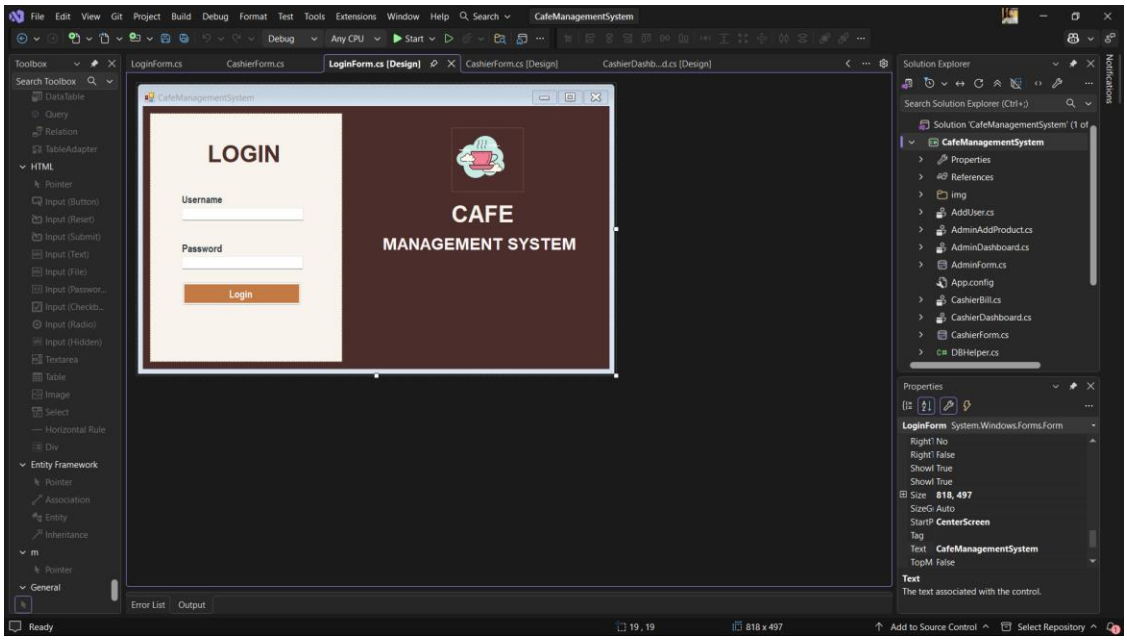


Figure 4: Login screen

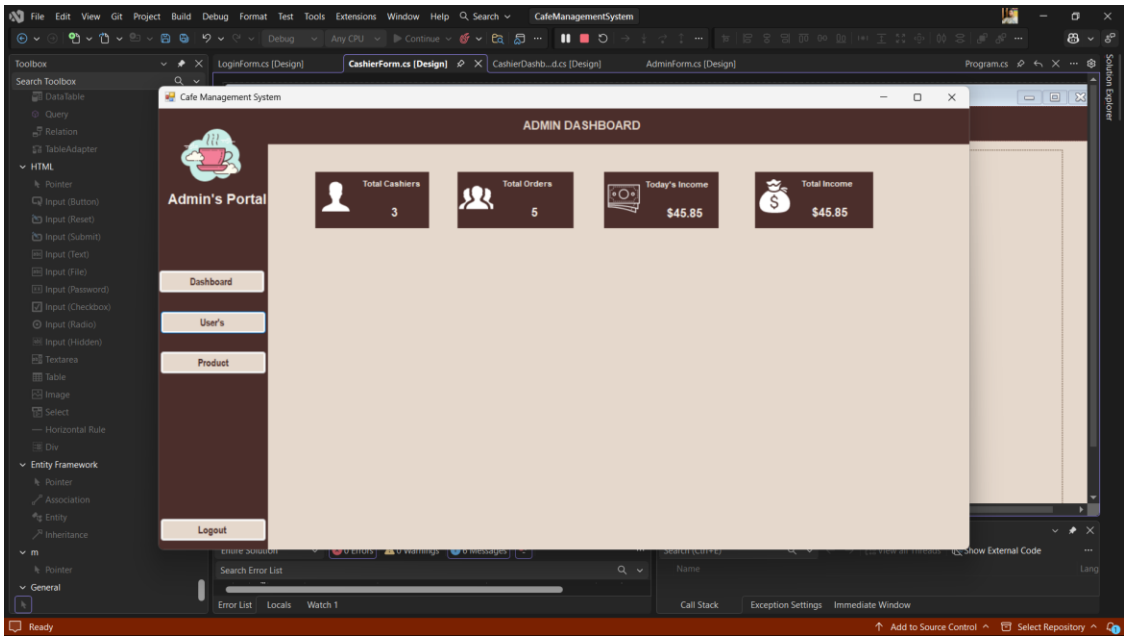


Figure 5: Admin Dashboard — store-wide statistics (Total Cashiers, Total Orders, Today's Income, Total Income)

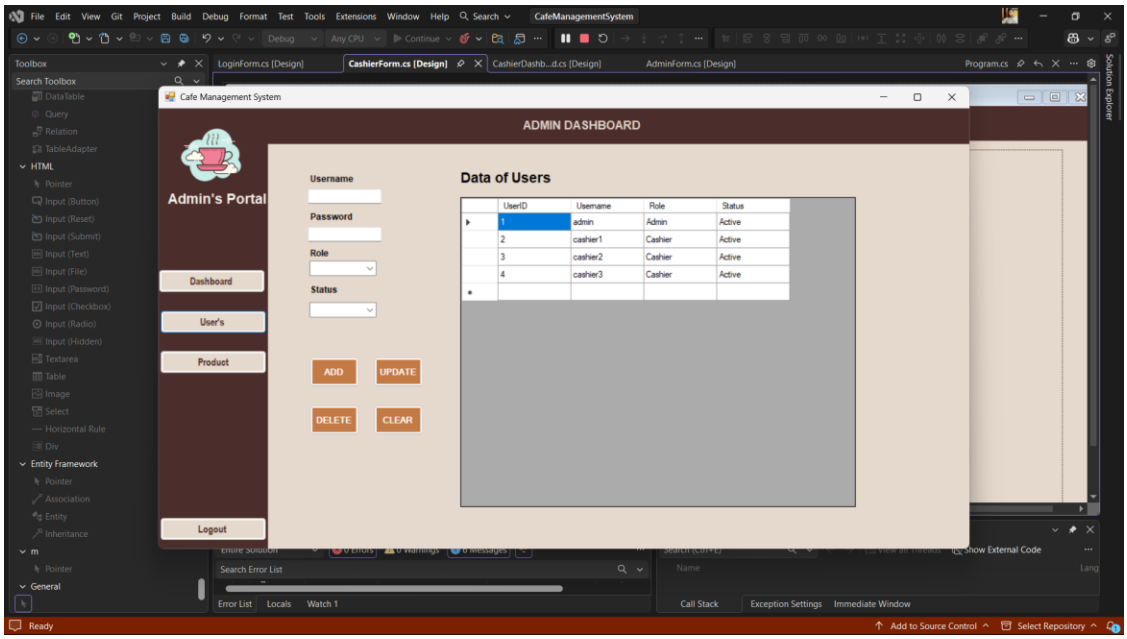


Figure 6: Admin — User Management screen (add/update/delete cashier and admin accounts)

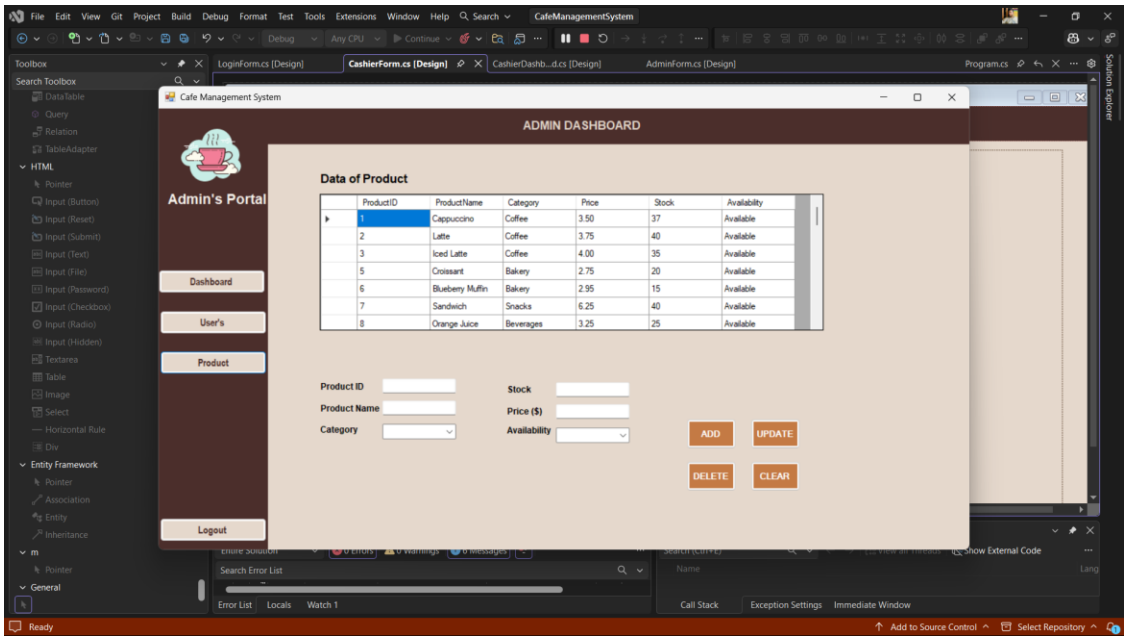


Figure 7: Admin — Product Management screen (add/update/delete menu items)

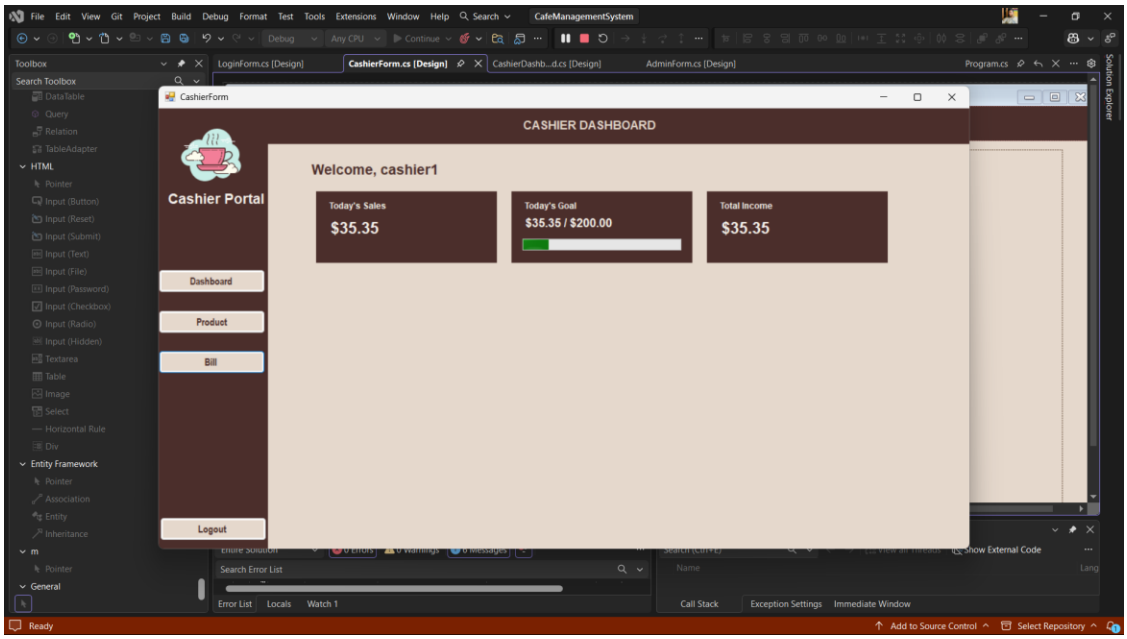


Figure 8: Cashier Dashboard — personal Today's Sales, Today's Goal progress, and Total Income

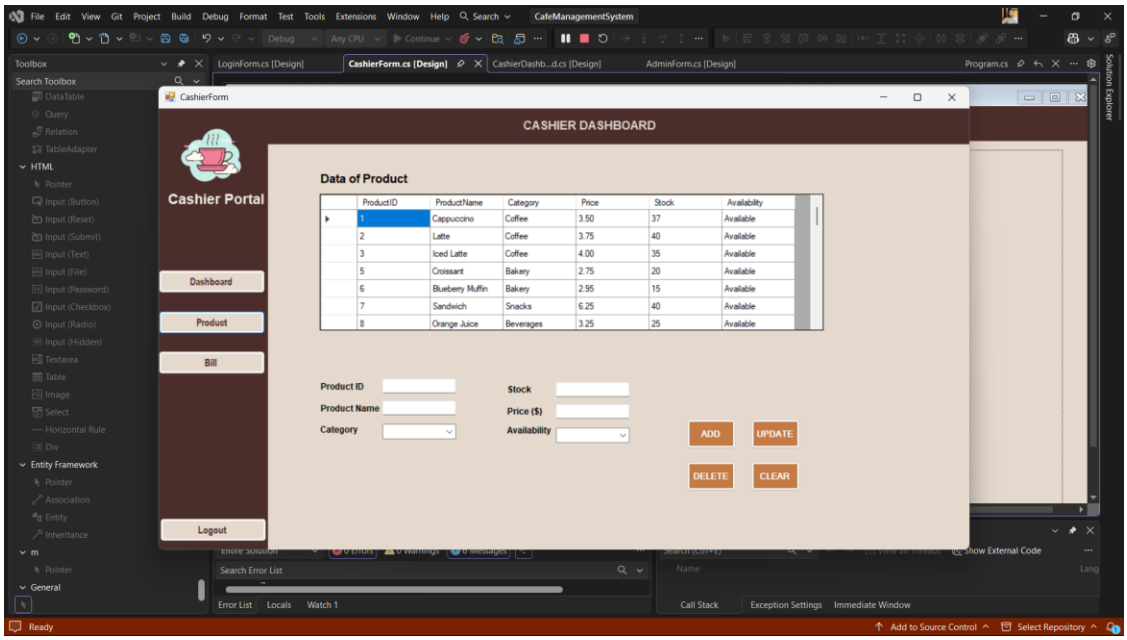


Figure 9: Cashier — Product view (read-only menu reference)

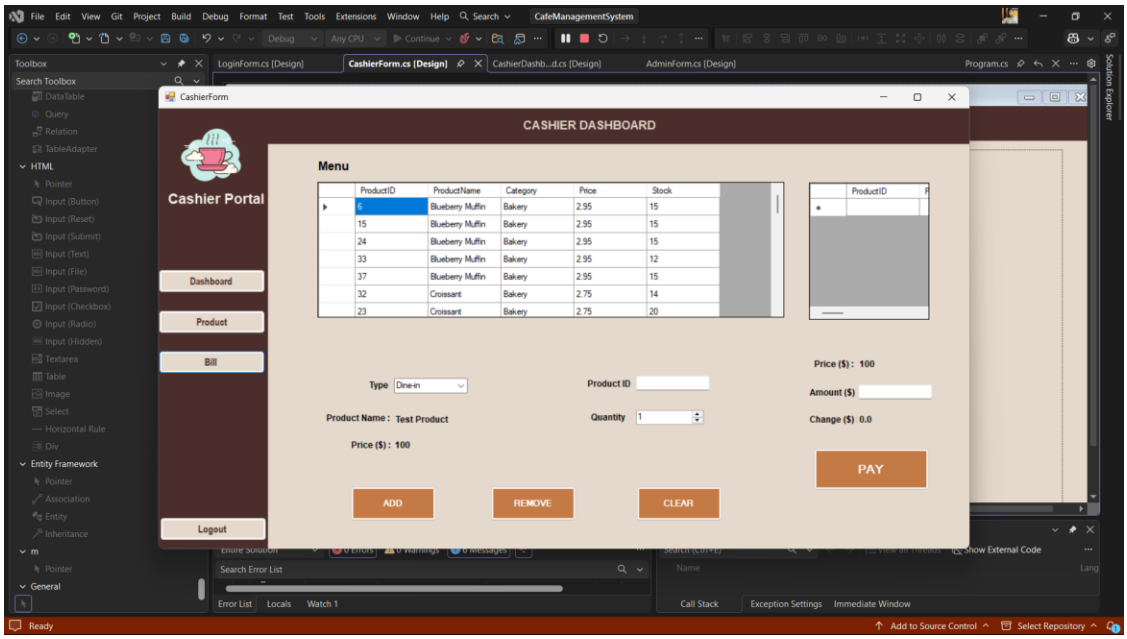


Figure 10: Cashier — Billing / POS screen (menu, cart, payment, and change calculation)